Content of src/app/api/file-tree/route.ts

```
import { NextResponse } from 'next/server';
import { buildFileTree } from '@/server/file-system';
import path from 'path';

// AÄ0D HD CD" tUB 4C´O Cô>C´CDt5CÔ8Dò 4CT@CT2C  DC 9C´>C" 1CT7 CÔ0D BD >CT: DD8C´LD$@C FC„8
export async function GET() {
  try {
    // Aô>C´CDt0CT< C¤0D$0C´>C2Â >D$:D44C  1D´;C  7C ?D4ICT=C  :Cä<C =CD0 (Cô@Cä5C¤B Cô>C´LCt>C$0D$5C
    // A„ACô>C´LCtCCT< Cô5D 5CÄ5CÔ=D4N Cä:D CCd5CÔ8DòÂ :CäBCä@D4N D4AD$0CÔ>C$8D" 7F  t.js
    const projectDir = process.env.USER_PROJECT_DIR || process.cwd();

    // B BD >C„< CD5D 5C$> DD0C";Cä2 C 5Cr ACô5Dd8C ;DÄ=D´E CÔ0D BD >CT:
    const fileTree = await buildFileTree(projectDir);

    return NextResponse.json({ fileTree });
  } catch (error) {
    console.error('Error getting file tree:', error);
    return NextResponse.json({ error: 'Failed to get file tree' }, { status: 500 });
  }
}

// AÄ0D HD CD"  õ5B 4C´O Cô>C´CDt5CÔ8Dò 4CT@CT2C  DC 9C´>C" A CÔ0D BD >C":C <C, DC„;DÄBD 0Dd8C€
export async function POST(request: Request) {
  try {
    // Aô>C´CDt0CT< CÔ0D BD >C":C, DC„;DÄBD 0Dd8C, 8Cr 7C ?D >D 0
    const { filterSettings } = await request.json();

    // Aô>C´CDt0CT< C¤0D$0C´>C2Â >D$:D44C  1D´;C  7C ?D4ICT=C  :Cä<C =CD0 (Cô@Cä5C¤B Cô>C´LCt>C$0D$5C
    const projectDir = process.env.USER_PROJECT_DIR || process.cwd();

    // B BD >C„< CD5D 5C$> DD0C";Cä2 D  CDt5D$>CÂ =C AD$@Cä5C
    const fileTree = await buildFileTree(projectDir, filterSettings);

    return NextResponse.json({ fileTree });
  } catch (error) {
    console.error('Error getting file tree:', error);
    return NextResponse.json({ error: 'Failed to get file tree' }, { status: 500 });
  }
}
```

Content of src/components/ui/use-toast.ts

```
"use client"

import * as React from "react"

import type {
  ToastActionElement,
  ToastProps,
} from "@/components/ui/toast"

const TOAST_LIMIT = 5
const TOAST_REMOVE_DELAY = 1000000

type ToasterToast = ToastProps & {
  id: string
  title?: React.ReactNode
  description?: React.ReactNode
  action?: ToastActionElement
}

const actionTypes = {
  ADD_TOAST: "ADD_TOAST",
  UPDATE_TOAST: "UPDATE_TOAST",
  DISMISS_TOAST: "DISMISS_TOAST",
  REMOVE_TOAST: "REMOVE_TOAST",
} as const

let count = 0

function genId() {
  count = (count + 1) % Number.MAX_SAFE_INTEGER
  return count.toString()
}

type ActionType = typeof actionTypes

type Action =
  | {
      type: ActionType["ADD_TOAST"]
      toast: ToasterToast
    }
  | {
      type: ActionType["UPDATE_TOAST"]
      toast: Partial<ToasterToast>
    }
  | {
      type: ActionType["DISMISS_TOAST"]
      toastId?: string
    }
  | {
      type: ActionType["REMOVE_TOAST"]
      toastId?: string
    }
```

```
interface State {
  toasts: ToasterToast[]
}

const toastTimeouts = new Map<string, ReturnType<typeof setTimeout>>()

const reducer = (state: State, action: Action): State => {
  switch (action.type) {
    case actionTypes.ADD_TOAST:
      return {
        ...state,
        toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
      }

    case actionTypes.UPDATE_TOAST:
      return {
        ...state,
        toasts: state.toasts.map((t) =>
          t.id === action.toast.id ? { ...t, ...action.toast } : t
        ),
      }

    case actionTypes.DISMISS_TOAST: {
      const { toastId } = action

      // ! Side effects ! - This could be extracted into a dismissToast() action,
      // but I'll keep it here for simplicity
      if (toastId) {
        toastTimeouts.set(
          toastId,
          setTimeout(() => {
            toastTimeouts.delete(toastId)
            dispatch({
              type: actionTypes.REMOVE_TOAST,
              toastId,
            })
          }, TOAST_REMOVE_DELAY)
        )
      }

      return {
        ...state,
        toasts: state.toasts.map((t) =>
          t.id === toastId || toastId === undefined
            ? {
                ...t,
                open: false,
              }
            : t
        ),
      }
    }
    case actionTypes.REMOVE_TOAST:
      if (action.toastId === undefined) {
        return {
          ...state,
```

```
          toasts: [],
        }
      }
      return {
        ...state,
        toasts: state.toasts.filter((t) => t.id !== action.toastId),
      }
  }
}

const listeners: Array<(state: State) => void> = []

let memoryState: State = { toasts: [] }

function dispatch(action: Action) {
  memoryState = reducer(memoryState, action)
  listeners.forEach((listener) => {
    listener(memoryState)
  })
}

type Toast = Omit<ToasterToast, "id">

function toast({ ...props }: Toast) {
  const id = genId()

  const update = (props: ToasterToast) =>
    dispatch({
      type: actionTypes.UPDATE_TOAST,
      toast: { ...props, id },
    })
  const dismiss = () => dispatch({ type: actionTypes.DISMISS_TOAST, toastId: id })

  dispatch({
    type: actionTypes.ADD_TOAST,
    toast: {
      ...props,
      id,
      open: true,
      onOpenChange: (open) => {
        if (!open) dismiss()
      },
    },
  })

  return {
    id,
    dismiss,
    update,
  }
}

function useToast() {
  const [state, setState] = React.useState<State>(memoryState)

  React.useEffect(() => {
```

```
    listeners.push(setState)
    return () => {
      const index = listeners.indexOf(setState)
      if (index > -1) {
        listeners.splice(index, 1)
      }
    }
  }, [state])

  return {
    ...state,
    toast,
    dismiss: (toastId?: string) => dispatch({ type: actionTypes.DISMISS_TOAST, toastId }),
  }
}

export { useToast, toast }
```